

MLP

- 矩阵 $\mathbf{X} \in \mathbb{R}^{n \times d}$ 来表示 n 个样本的小批量，其中每个样本具有 d 个输入特征
- 对于具有 h 个隐藏单元的单隐藏层多层感知机，用 $\mathbf{H} \in \mathbb{R}^{n \times h}$ 表示隐藏层的输出
- 隐藏层权重 $\mathbf{W}^{(1)} \in \mathbb{R}^{d \times h}$
- 隐藏层偏置 $\mathbf{b}^{(1)} \in \mathbb{R}^{1 \times h}$
- 输出层权重 $\mathbf{W}^{(2)} \in \mathbb{R}^{h \times q}$
- 输出层偏置 $\mathbf{b}^{(2)} \in \mathbb{R}^{1 \times q}$
- 非线性的激活函数 (activation function) σ
- 单隐藏层多层感知机的输出 $\mathbf{O} \in \mathbb{R}^{n \times q}$:

$$\mathbf{H} = \sigma(\mathbf{XW}^{(1)} + \mathbf{b}^{(1)}),$$
$$\mathbf{O} = \mathbf{HW}^{(2)} + \mathbf{b}^{(2)}.$$

 一个单隐藏层的多层感知机，具有5个隐藏单元

激活函数

ReLU函数

- 分段，取0和x的最大值
- 导数：要么参数消失，要么通过
 - ≤ 0 : 导数=0
 - > 0 : 导数=1

$$\text{ReLU}(x) = \max(x, 0).$$

sigmoid函数

- 将输入变换到(0,1)上的输出
- 平滑，输入接近0时sigmoid函数接近线性变换
- 导数
 - 关于0单峰对称
 - 0处达最大值0.25
 - 无穷处=0

$$\text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}.$$

tanh函数

- 即双曲正切函数，将区间压缩到(-1,1)
- 在0附近时，接近于线性变换
- 导数
 - 关于0单峰对称
 - 0处达最大值1
 - 无穷处=0

$$\tanh(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}.$$



代码

以MNIST的28x28的10类照片的分类为例

- 层宽度：通常取2的若干次幂
 - 原因：内存在硬件中的分配和寻址方式

新代码

- 搭建模型

```
net = nn.Sequential(nn.Flatten(),
                    nn.Linear(784,256),
                    nn.ReLU(),
                    nn.Linear(256,10))

def init_weight(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight,std=0.01)

net.apply(init_weights)
```

过拟合

统计学习理论

- 独立同分布假设：假设训练数据和测试数据都是从相同分布独立提取
- 现实：常常违背该假设

模型选择

- 采用验证集，但其与测试集的边界十分模糊
- K折交叉验证，即训练数据分成K份，进行K次训练和验证，每次有K-1份用于训练

权重衰减

L_2 正则化——岭回归

- 背景：以多项式回归为例，阶数越大，复杂性越高，需要一个工具来调整复杂性
- 操作：
 - 添加一个正则化项，包含正则化常数 λ 和 $\|\mathbf{w}\|^2$
 - 从“最小化 $L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)})^2$.”
 - 到最小化“ $L(\mathbf{w}, b) + \frac{\lambda}{2} \|\mathbf{w}\|^2$,”
- 对权重向量的大分量施加了巨大惩罚
- 使学习算法偏向于在大量特征上均匀分布权重的模型
- 在单个变量中的观测误差更稳定
- 小批量随机梯度下降从原来的“ $\mathbf{w}_{\text{new}} = \mathbf{w} - \eta * \text{梯度}$ ”变为“ $\mathbf{w}_{\text{new}} = (1 - \eta\lambda) * \mathbf{w} - \eta * \text{梯度}$ ”
- 由于 $\mathbf{w}$ 每次更新前先缩小，所以称作**权重衰减**

$$\mathbf{w} \leftarrow (1 - \eta\lambda) \mathbf{w} - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \mathbf{x}^{(i)} (\hat{y} - y^{(i)}) .$$

L_1 正则化——套索回归

- 导致模型将权重集中在一小部分特征上，其余的清楚为0
- 称作 **特征选择**，部分场景需要

暂退法Dropout

权重衰减认为参数的范数是模型“简单性”的度量之一。而“简单性”的另一角度是平滑性，即不应该对微小变化敏感，那么即使我注入噪声，如随机丢弃dropout一些神经元，理应结果不变。

简洁实现

依旧以MNIST照片为例

```

dropout1 = 0.2
dropout2 = 0.5

net = nn.Sequential(nn.Flatten(),
                    nn.Linear(784,256),
                    nn.ReLU(),
                    # 在第一个全连接层之后添加一个dropout层
                    nn.Dropout(dropout1), # 抛弃概率为dropout1
                    nn.Linear(256,256),
                    nn.ReLU(),
                    # 在第二个全连接层之后添加一个dropout层
                    nn.Dropout(dropout2), # 抛弃概率为dropout2
                    nn.Linear(256,10))

```

传播与计算图

前向传播

为了简单起见，我们假设输入样本是 $\mathbf{x} \in \mathbb{R}^d$ ，
并且我们的隐藏层不包括偏置项。
这里的中间变量是：

$$\mathbf{z} = \mathbf{W}^{(1)}\mathbf{x},$$

其中 $\mathbf{W}^{(1)} \in \mathbb{R}^{h \times d}$

是隐藏层的权重参数。

将中间变量 $\mathbf{z} \in \mathbb{R}^h$ 通过激活函数 ϕ 后，
我们得到长度为 h 的隐藏激活向量：

$$\mathbf{h} = \phi(\mathbf{z}).$$

隐藏变量 $\mathbf{h}$ 也是一个中间变量。

假设输出层的参数只有权重 $\mathbf{W}^{(2)} \in \mathbb{R}^{q \times h}$ ，
我们可以得到输出层变量，它是一个长度为 q 的向量：

$$\mathbf{o} = \mathbf{W}^{(2)}\mathbf{h}.$$

假设损失函数为 l ，样本标签为 y ，我们可以计算单个数据样本的损失项，

$$L = l(\mathbf{o}, y).$$

根据 L_2 正则化的定义，给定超参数 λ ，正则化项为

$$s = \frac{\lambda}{2} \left(\|\mathbf{W}^{(1)}\|_F^2 + \|\mathbf{W}^{(2)}\|_F^2 \right),$$

:eqlabel: eq_forward-s

其中矩阵的Frobenius范数是将矩阵展平为向量后应用的 L_2 范数。

最后，模型在给定数据样本上的正则化损失为：

$$J = L + s.$$

在下面的讨论中，我们将 J 称为*目标函数*（objective function）。

 alt text

 前向传播的计算图

反向传播

根据链式法则而得，可以得知：

- $\frac{\partial J}{\partial \mathbf{W}^{(2)}}$ 与 λ 和 h^T 有关
- $\frac{\partial J}{\partial \mathbf{W}^{(1)}}$ 与 λ 和 x^T 有关

 alt text

训练网络

- 前向传播：正则项取决于 $\mathbf{W}^{(1)}$ 和 $\mathbf{W}^{(2)}$
- 反向传播：梯度计算取决于前向传播给出的 h 以及最初的 x

因此，训练时，在初始化模型参数后，应该交替前向和反向传播，利用反向传播给出的梯度来更新模型参数。反向传播会重复利用前向传播存储的中间值，因此需要保留中间值，也因此需要更多内存。中间值大小与网络层数量批量大小大致呈正比，所以大批量数据训练更深层次网络更容易导致内存不足，即 Out of Memory

数值稳定性和模型初始化

梯度爆炸和消失

考虑一个具有 L 层、输入 $\mathbf{x}$ 和输出 $\mathbf{o}$ 的深层网络。

每一层 l 由变换 f_l 定义，

该变换的参数为权重 $\mathbf{W}^{(l)}$ ，

其隐藏变量是 $\mathbf{h}^{(l)}$ （令 $\mathbf{h}^{(0)} = \mathbf{x}$ ）。

我们的网络可以表示为：

$$\mathbf{h}^{(l)} = f_l(\mathbf{h}^{(l-1)}) \text{ 因此 } \mathbf{o} = f_L \circ \dots \circ f_1(\mathbf{x}).$$

如果所有隐藏变量和输入都是向量，
我们可以将 $\mathbf{o}$ 关于任何一组参数 $\mathbf{W}^{(l)}$ 的梯度写为下式：

$$\partial_{\mathbf{W}^{(l)}} \mathbf{o} = \underbrace{\partial_{\mathbf{h}^{(L-1)}} \mathbf{h}^{(L)}}_{\mathbf{M}^{(L)} \stackrel{\text{def}}{=}} \cdot \dots \cdot \underbrace{\partial_{\mathbf{h}^{(l)}} \mathbf{h}^{(l+1)}}_{\mathbf{M}^{(l+1)} \stackrel{\text{def}}{=}} \underbrace{\partial_{\mathbf{W}^{(l)}} \mathbf{h}^{(l)}}_{\mathbf{v}^{(l)} \stackrel{\text{def}}{=}}.$$

换言之，该梯度是 $L - l$ 个矩阵

$$\mathbf{M}^{(L)} \cdot \dots \cdot \mathbf{M}^{(l+1)}$$

与梯度向量 $\mathbf{v}^{(l)}$ 的乘积。

梯度消失

- 参数更新过小，更新时几乎无法移动，导致模型无法学习
- sigmoid函数常常导致梯度消失，因为在输入很大或很小时输出接近于0

梯度爆炸

- 若梯度爆炸是初始化导致的（如选择标准差为1，而非常见的0.01），不可能让梯度下降优化器收敛

打破对称性

如果所有参数初始化都为常数，即具有对称性的话，则前向传播会产生相同激活，送到输出；反向传播会得到相同梯度，因此迭代之后参数仍相同，始终无法打破对称性，参数带来的效果大打折扣。不过，Dropout可以打破这种对称性

参数初始化

即解决或减轻上述对称性带来的问题

某些没有非线性的全连接层输出（例如，隐藏变量） o_i 的尺度分布。

对于该层 n_{in} 输入 x_j 及其相关权重 w_{ij} ，输出由下式给出

$$o_i = \sum_{j=1}^{n_{\text{in}}} w_{ij} x_j.$$

默认初始化

- 正态分布
- 输入越多，输出方差就越容易消失或爆炸
- 即深层的网络的方差容易受到层数影响

Xavier初始化

- 权重 w_{ij} 都是从同一分布中独立抽取的，假设该分布具有零均值和方差 σ^2 （不意味着这是高斯）
- 假设层 x_j 的输入也具有零均值和方差 γ^2 ，并且它们独立于 w_{ij} 并且彼此独立。
- 计算 o_i 的方差（均值为0）：

$$\begin{aligned}\text{Var}[o_i] &= E[o_i^2] - (E[o_i])^2 \\ &= \sum_{j=1}^{n_{\text{in}}} E[w_{ij}^2 x_j^2] - 0 \\ &= \sum_{j=1}^{n_{\text{in}}} E[w_{ij}^2] E[x_j^2] \\ &= n_{\text{in}} \sigma^2 \gamma^2.\end{aligned}$$

- 保持方差不变的话，需设置 $n_{\text{in}} \sigma^2 = 1$ ，同理，反向的话需设置 $n_{\text{out}} \sigma^2 = 1$ ，否则方差会和 n_{in} 和 n_{out} 强相关。因此，我们需满足：

$$\frac{1}{2}(n_{\text{in}} + n_{\text{out}}) \sigma^2 = 1 \text{ 或等价于 } \sigma = \sqrt{\frac{2}{n_{\text{in}} + n_{\text{out}}}}.$$

- 通常，Xavier初始化从均值为零，方差 $\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}$ 的高斯分布中采样权重。
- 当然，也可以使用均匀分布，从其方差入手，可以知道应该从 $U\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$ 中抽取权重

环境和分布偏移

核心问题：训练时用的数据和实际用的时候数据不一样了怎么办？

偏移类型

协变量偏移——输入变了

- 由于协变量（即特征）分布变化而产生的
- 假设：
 - 虽然输入的分布可能随时间而**改变**
 - 但标签函数（即条件分布 $P(y | \mathbf{x})$ ）没有改变。
- 以猫狗分类为例
 - 训练时，使用真实照片
 - 测试时，标签没变，但是使用卡通照片，**改变了输入**
- **纠正**
 - 给训练样本加权

- 对训练集里和测试数据更像的样本，赋予更大权重，让训练数据更接近测试分布

标签偏移——输出变了

- 假设
 - 标签边缘概率 $P(y)$ 改变
 - 但是类别条件分布 $P(\mathbf{x} | y)$ 在不同的领域之间保持不变。
- 以预测患者疾病为例
 - 训练和测试时，病人比例变化了，即 $P(y)$
 - 但是疾病引起的症状依旧不变，即 $P(\mathbf{x} | y)$
- 更经典的例子
 - 如果只有0.01%的病人，其余的都是正常人
 - 那么训练时，模型会偏向于预测所有人都是正常人
 - 但是测试时，病人远不止0.01%，因此会误判病人为正常人
- 纠正
 - 用混淆矩阵修正
 - 先用旧模型对测试集预测，反推出测试集的真实标签分布，据此**调整权重**
 - 例如病人少的话，就给病人多加点权重

概念偏移——定义变了

- 例子
 - 训练：2019 年 "流行音乐" = 某种风格
 - 测试：2024： "流行音乐" 的定义已经完全不同了
 - 概念变了
- 纠正
 - 持续更新模型
 - 无法用数学修正，只能不断用新数据重新训练或微调

偏移示例

医学诊断

- 使用大学生数据进行训练，模型容易受到大学生本身的影响，在真实患者那有偏差
- **协变量偏移**

自动驾驶汽车

- 用游戏合成数据来训练，但在真实测试中，所有路沿都被渲染成简单的纹理
- **协变量偏移**

非平稳分布

分布变化缓慢且模型没有充分更新时

- 计算广告模型，但只知道09年前的知识
- 垃圾邮件过滤器，会让垃圾邮件发送者制造出没用于训练的垃圾邮件
- 推荐圣诞帽系统，但是一年四季都推荐

学习问题的分类法

核心：环境会不会变、会不会记住你的行为，决定了你该用哪种学习方法。

类型	特点	例子
批量学习	模型训练一次，部署后不再更新	猫狗分类器，装到猫门上就不管了
在线学习	每来一个数据，模型就更新一次	股票预测，每天根据结果调整
老虎机	只有有限个选择，探索 vs 利用	推荐哪条广告点击率最高
控制	环境会记住你之前的操作	恒温器：之前加热了，现在温度就会高
强化学习	智能体通过试错学习最优策略	AlphaGo、自动驾驶

公平、责任和透明度

机器学习模型的伦理问题：

- 精度不是唯一标准
 - 诊断出错分两种：把健康人当病人 vs 把病人当健康人
 - 前者多花点钱，后者可能要命 -> 成本不一样
- 反馈循环
 - 警察去犯罪率高的地区巡逻 -> 那里被抓的人更多 -> 数据更多 -> 模型说"这里犯罪率高" -> 更多警察去... 形成恶性循环
- 算法歧视
 - 模型可能会因为训练数据里的偏见，对某些群体不公平